

INTELLIPAAT

MODULE 2 - MODERN JAVASCRIPT FOR BACKEND DEVELOPMENT

BACKEND DEVELOPMENT ASSIGNMENT

Name	SAGAR KUMAR
PRN	SOE25BTAM29
Section	A

Table of Contents

- 1. Question 1 — Modern JavaScript
 - 2. Question 2 — Modular Backend Application
 - 3. Question 3 — Event Loop & Asynchronous Execution
 - 4. Question 4 — Reliable Asynchronous Application
 - 5. Question 5 — Mini Project: Modular Backend Utility
 - 6. Testing & Execution
 - 7. Conclusion
-

Environment Used

Item	Details
Runtime	Node.js (tested on v22 LTS; works on v18 and above)
Language	JavaScript (ES6+), ES Modules
External packages	dotenv (Question 5 only)
Editor	Visual Studio Code
Operating systems supported	Windows, macOS, Linux

1. Question 1 — Modern JavaScript

Title: Refactoring a Legacy Order-Billing Module using ES6+ Features

Aim / Objective

To refactor a small part of a legacy Node.js application (an order-billing module) written in old JavaScript syntax into modern JavaScript (ES6+), and to explain how `let / const` , arrow functions, template literals and other modern features improve readability, maintainability and reliability.

Approach

- 1. Take a small legacy module (`legacy.js`) that prints a billing report for customer orders. It uses `var` , `function` expressions, string concatenation with `+` , manual `for` loops and `||` for default values.
- 2. Rewrite the same logic in `app.js` using modern syntax, without changing the business result (the totals must stay the same).
- 3. Add one extra order with missing data to show that the modern version is also safer.
- 4. Run both versions and compare the output.

Files

File	Purpose
<code>legacy.js</code>	The original legacy code (problem / before refactoring)
<code>app.js</code>	The refactored modern JavaScript version (solution)
<code>package.json</code>	Marks the folder as an ES Module project and adds <code>npm start</code>

Code

Problem — legacy approach (`legacy.js`)

```
// QUESTION 1 - LEGACY VERSION (before refactoring)
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A
//
// This is the old style code taken from a legacy order-billing module.
// Problems: `var` is function-scoped and can be re-declared by mistake,
// string concatenation with + is hard to read, every callback needs the
// long `function` keyword, and default values are handled manually with ||.

var TAX_RATE = 0.18;

var orders = [
  { id: 101, customer: "Aman", items: [{ name: "Keyboard", price: 750, qty: 2 }, { name:
"Mouse", price: 400, qty: 1 }] },
  { id: 102, customer: "Priya", items: [{ name: "Monitor", price: 8500, qty: 1 }] },
  { id: 103, customer: "Rohit", items: [{ name: "USB Cable", price: 150, qty: 3 }], discount:
10 }
];
```

```

function calculateSubtotal(items) {
  var total = 0;
  for (var i = 0; i < items.length; i++) {
    total = total + items[i].price * items[i].qty;
  }
  return total;
}

function generateBill(order) {
  var discount = order.discount || 0;
  var subtotal = calculateSubtotal(order.items);
  var afterDiscount = subtotal - (subtotal * discount) / 100;
  var tax = afterDiscount * TAX_RATE;
  var finalAmount = afterDiscount + tax;

  return "Order #" + order.id + " | Customer: " + order.customer +
    " | Subtotal: Rs. " + subtotal.toFixed(2) +
    " | Discount: " + discount + "%" +
    " | Tax: Rs. " + tax.toFixed(2) +
    " | Total: Rs. " + finalAmount.toFixed(2);
}

console.log("=== Legacy Billing Report ===");
for (var j = 0; j < orders.length; j++) {
  console.log(generateBill(orders[j]));
}

var highValue = orders.filter(function (order) {
  return calculateSubtotal(order.items) > 1000;
});
var names = [];
for (var k = 0; k < highValue.length; k++) {
  names.push(highValue[k].customer);
}
console.log("High value customers: " + names.join(", "));

```

Problems in the legacy code

- `var` is function-scoped, can be re-declared silently, and values that should never change (like `TAX_RATE`) can be overwritten by mistake.
- Long string concatenation with `+` and `" | "` is hard to read and easy to break.
- Manual `for` loops with index variables (`i`, `j`, `k`) add noise and can cause off-by-one errors.
- `order.discount || 0` treats a valid value `0` the same as "missing", and if `order.items` is missing the program crashes.

Solution — refactored modern code (app.js)

```

// QUESTION 1 - MODERN JAVASCRIPT (ES6+) REFACTORED VERSION
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A
//
// This file is the refactored version of legacy.js (order-billing module).
// Features used:
//   1. let and const
//   2. Arrow functions
//   3. Template literals
//   4. Extra ES6+ features: destructuring, default parameters,
//      array methods (reduce/filter/map), optional chaining (?),
//      nullish coalescing (??) and Object.freeze()

// const -> this value must never change, so reassigning it causes an error.

```

```

// Object.freeze() also stops properties from being changed accidentally.
const CONFIG = Object.freeze({
  TAX_RATE: 0.18,
  HIGH_VALUE_LIMIT: 1000,
  CURRENCY: 'Rs.',
});

const orders = [
  {
    id: 101,
    customer: 'Aman',
    items: [
      { name: 'Keyboard', price: 750, qty: 2 },
      { name: 'Mouse', price: 400, qty: 1 },
    ],
  },
  { id: 102, customer: 'Priya', items: [{ name: 'Monitor', price: 8500, qty: 1 }] },
  { id: 103, customer: 'Rohit', items: [{ name: 'USB Cable', price: 150, qty: 3 }], discount:
10 },
  { id: 104, customer: 'Neha' }, // order with missing items (handled safely)
];

// Arrow function + destructuring of each item + reduce() instead of a for loop.
// Default parameter (items = []) protects against undefined input.
const calculateSubtotal = (items = []) =>
  items.reduce((total, { price, qty }) => total + price * qty, 0);

// Small helper that formats money in one place (easy to change later).
const formatMoney = (amount) => `${CONFIG.CURRENCY} ${amount.toFixed(2)}`;

const generateBill = (order) => {
  // Object destructuring with a default value for discount.
  const { id, customer, discount = 0 } = order;

  // Optional chaining (?.) + nullish coalescing (??):
  // if order.items is missing we use an empty array instead of crashing.
  const items = order?.items ?? [];

  const subtotal = calculateSubtotal(items);
  const afterDiscount = subtotal - (subtotal * discount) / 100;
  const tax = afterDiscount * CONFIG.TAX_RATE;
  const finalAmount = afterDiscount + tax;

  // Template literal: the whole line is readable in one go.
  return `Order #${id} | Customer: ${customer} | Items: ${items.length} | ` +
    `Subtotal: ${formatMoney(subtotal)} | Discount: ${discount}% | ` +
    `Tax: ${formatMoney(tax)} | Total: ${formatMoney(finalAmount)}`;
};

console.log('=== Modern Billing Report ===');
orders.forEach((order) => console.log(generateBill(order)));

// filter() + map() chain replaces the manual loop and temporary array.
const highValueCustomers = orders
  .filter(({ items }) => calculateSubtotal(items) > CONFIG.HIGH_VALUE_LIMIT)
  .map(({ customer }) => customer);

console.log(`High value customers: ${highValueCustomers.join(', ')}`);

// let is used only where the value really changes (block-scoped counter).
let totalRevenue = 0;
for (const order of orders) {
  totalRevenue += calculateSubtotal(order.items);
}
console.log(`Total revenue before tax: ${formatMoney(totalRevenue)}`);

// Demonstrating the reliability benefit of const + Object.freeze().
try {

```

```

    CONFIG = {}; // reassigning a const variable
  } catch (error) {
    console.log(`Protected by const: ${error.name} - ${error.message}`);
  }

  try {
    CONFIG.TAX_RATE = 0.5; // ES Modules run in strict mode, so this throws
  } catch (error) {
    console.log(`Protected by Object.freeze(): ${error.name} - ${error.message}`);
  }
  console.log(`TAX_RATE is still ${CONFIG.TAX_RATE}`);

```

Expected Output

Run the commands from inside the `question-1` folder (Node.js 18 or above).

Legacy version (for comparison):

```

$ node legacy.js
=== Legacy Billing Report ===
Order #101 | Customer: Aman | Subtotal: Rs. 1900.00 | Discount: 0% | Tax: Rs. 342.00 | Total: Rs. 2242.00
Order #102 | Customer: Priya | Subtotal: Rs. 8500.00 | Discount: 0% | Tax: Rs. 1530.00 | Total: Rs. 10030.00
Order #103 | Customer: Rohit | Subtotal: Rs. 450.00 | Discount: 10% | Tax: Rs. 72.90 | Total: Rs. 477.90
High value customers: Aman, Priya

```

Refactored modern version:

```

$ node app.js
=== Modern Billing Report ===
Order #101 | Customer: Aman | Items: 2 | Subtotal: Rs. 1900.00 | Discount: 0% | Tax: Rs. 342.00 | Total: Rs. 2242.00
Order #102 | Customer: Priya | Items: 1 | Subtotal: Rs. 8500.00 | Discount: 0% | Tax: Rs. 1530.00 | Total: Rs. 10030.00
Order #103 | Customer: Rohit | Items: 1 | Subtotal: Rs. 450.00 | Discount: 10% | Tax: Rs. 72.90 | Total: Rs. 477.90
Order #104 | Customer: Neha | Items: 0 | Subtotal: Rs. 0.00 | Discount: 0% | Tax: Rs. 0.00 | Total: Rs. 0.00
High value customers: Aman, Priya
Total revenue before tax: Rs. 10850.00
Protected by const: TypeError - Assignment to constant variable.
Protected by Object.freeze(): TypeError - Cannot assign to read only property 'TAX_RATE' of object '#<Object>'
TAX_RATE is still 0.18

```

The billing totals for orders 101-103 are identical in both versions, which proves the refactoring did not change the business logic. The modern version also handles order 104 (missing `items`) without crashing.

Explanation

Modern feature	Where it is used	How it helps
<code>const</code>	<code>CONFIG</code> , <code>orders</code> , all functions, <code>subtotal</code> , <code>tax</code>	A <code>const</code> variable cannot be reassigned. Values that should never change are protected — the output shows <code>TypeError: Assignment to constant variable</code> . A reader immediately knows the value is fixed. (Reliability, readability)
<code>let</code>	<code>totalRevenue</code>	Used only where the value really changes. <code>let</code> is block-scoped, so the variable does not leak outside the block like <code>var</code> does, and it cannot be re-declared in the same scope. (Reliability)
Arrow functions	<code>calculateSubtotal</code> , <code>formatMoney</code> , <code>generateBill</code> , all callbacks	Shorter syntax, implicit return for one-line functions, and callbacks such as <code>.map(({ customer }) => customer)</code> read like plain English. (Readability)
Template literals	Bill line, summary messages	Variables are written directly inside the string with <code>\${...}</code> . There is no need to open and close quotes and add <code>+</code> around every variable, so the final output format is easy to see and change. (Readability, maintainability)
Destructuring	<code>const { id, customer, discount = 0 } = order, ({ price, qty })</code>	Extracts only the needed properties in one line and documents which fields a function uses. (Readability)
Default parameters / values	<code>(items = []), discount = 0</code>	Missing values get a safe default automatically, instead of manual <code>`</code>
Optional chaining <code>?.</code> and nullish coalescing <code>??</code>	<code>order?.items ?? []</code>	If <code>items</code> is missing, an empty array is used and the program does not crash with <code>Cannot read properties of undefined</code> . Unlike <code>`</code>
Array methods <code>reduce</code> , <code>filter</code> , <code>map</code> , <code>forEach</code> , <code>for...of</code>	Subtotal and high-value customer list	Replace manual index loops and temporary arrays. The intent (“sum”, “filter”, “pick names”) is clear and there are no index bugs. (Maintainability)
<code>Object.freeze()</code>	<code>CONFIG</code>	Prevents properties of the configuration object from being changed at runtime. Because ES Modules run in strict mode, an accidental change throws an error instead of failing silently. (Reliability)

Conclusion

The refactored module produces the same billing results as the legacy code but is shorter, easier to read and safer. `const` and `let` prevent accidental reassignment and scope leaks, arrow functions and array methods remove boilerplate, and template literals make output

strings readable. Additional ES6+ features such as destructuring, default values, optional chaining and nullish coalescing protect the application from missing data, which makes the code more reliable and easier to maintain in the future.

2. Question 2 — Modular Backend Application

Title: Student Result Management Application using ES Modules

Aim / Objective

To redesign a single-file Node.js student application into a modular application using ES Modules (`import` / `export`), with a separate data module, a reusable utility/logic module and a main application module, and to show that the imported functionality works correctly.

Approach

Problem (before): the original program kept the student array, the calculation functions (average, grade, topper, search) and the printing logic in one file. Any change — even adding one student — meant editing the same large file, and the functions could not be reused in another program.

Solution (after): the code is split by responsibility:

1. **Data module** (`src/data/students.js`) — only stores the student records and the `PASS_MARKS` constant.
2. **Utility / logic module** (`src/utils/studentUtils.js`) — reusable, pure functions that work on any list of students.
3. **Main application module** (`src/app.js`) — imports the data and the functions and runs the program.
4. `package.json` contains `"type": "module"`, so Node.js treats every `.js` file as an ES Module and allows `import/export`.

Folder structure

```
question-2/
├── src/
│   ├── data/
│   │   └── students.js      # Data module (named exports)
│   ├── utils/
│   │   └── studentUtils.js  # Logic module (named exports + default export)
│   └── app.js               # Main application module (entry point)
├── package.json             # "type": "module" enables ES Modules
└── README.md
```

Explanation of every important file

File	Role
<code>package.json</code>	Project metadata. <code>"type": "module"</code> enables ES Module syntax. <code>npm start</code> runs <code>node src/app.js</code> .

File	Role
src/data/students.js	Exports <code>students</code> (array of records) and <code>PASS_MARKS</code> using named exports . Contains no logic, so data can later be replaced by a database without touching other files.
src/utils/studentUtils.js	Imports <code>PASS_MARKS</code> from the data module and exports reusable functions: <code>getAverage</code> , <code>getGrade</code> , <code>hasPassed</code> , <code>findStudentById</code> , <code>filterByBranch</code> , <code>getTopper</code> , <code>formatStudent</code> (named exports) and <code>buildReport</code> (default export).
src/app.js	Entry point. Imports the data and the functions, then prints a full report, the topper, CSE students, pass/fail count and a search by ID (found and not found).

Code

package.json

```
{
  "name": "question-2-student-modules",
  "version": "1.0.0",
  "description": "Module 2 - Question 2: Modular student management application using ES
Modules",
  "type": "module",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js"
  },
  "author": "SAGAR KUMAR (SOE25BTAM29)",
  "license": "ISC"
}
```

src/data/students.js — data module

```
// Data module: holds only the student records and related constants.
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A

// Named export - other modules import exactly what they need.
export const PASS_MARKS = 40;

export const students = [
  { id: 1, name: 'Aarav Sharma', branch: 'CSE', marks: { maths: 88, physics: 76, programming:
95 } },
  { id: 2, name: 'Diya Patel', branch: 'CSE', marks: { maths: 92, physics: 89, programming:
90 } },
  { id: 3, name: 'Kabir Singh', branch: 'ECE', marks: { maths: 35, physics: 42, programming:
38 } },
  { id: 4, name: 'Ananya Gupta', branch: 'ME', marks: { maths: 67, physics: 71, programming:
60 } },
  { id: 5, name: 'Rohan Verma', branch: 'CSE', marks: { maths: 55, physics: 48, programming:
72 } },
];
```

src/utils/studentUtils.js — utility / logic module

```
// Utility / logic module: reusable functions that work on student data.
// These functions do not depend on where the data comes from, so they can
```

```

// be reused with any list of students (file, database, API, etc.).
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A

import { PASS_MARKS } from '../data/students.js';

// Average of all subject marks for one student.
export const getAverage = ({ marks }) => {
  const scores = Object.values(marks);
  const total = scores.reduce((sum, score) => sum + score, 0);
  return Number((total / scores.length).toFixed(2));
};

// Converts an average into a grade.
export const getGrade = (average) => {
  if (average >= 90) return 'A+';
  if (average >= 75) return 'A';
  if (average >= 60) return 'B';
  if (average >= PASS_MARKS) return 'C';
  return 'F';
};

// A student passes only if every subject is at or above PASS_MARKS.
export const hasPassed = ({ marks }) =>
  Object.values(marks).every((score) => score >= PASS_MARKS);

export const findStudentById = (students, id) =>
  students.find((student) => student.id === id) ?? null;

export const filterByBranch = (students, branch) =>
  students.filter((student) => student.branch === branch);

export const getTopper = (students) =>
  students.reduce((best, current) =>
    getAverage(current) > getAverage(best) ? current : best);

export const formatStudent = (student) => {
  const average = getAverage(student);
  const status = hasPassed(student) ? 'PASS' : 'FAIL';
  return `${String(student.id).padEnd(3)} ${student.name.padEnd(14)}
${student.branch.padEnd(5)} ` +
    `Avg: ${average.toFixed(2).padStart(6)} Grade: ${getGrade(average).padEnd(3)}
${status}`;
};

// Default export - the main helper of this module.
const buildReport = (students) => students.map(formatStudent).join('\n');

export default buildReport;

```

src/app.js — main application module

```

// Main application module: imports data and logic, then runs the program.
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A

import { students, PASS_MARKS } from '../data/students.js';
import buildReport, {
  getAverage,
  getGrade,
  hasPassed,
  findStudentById,
  filterByBranch,
  getTopper,
} from '../utils/studentUtils.js';

const line = '-'.repeat(60);

```

```

console.log('STUDENT RESULT MANAGEMENT (ES Modules Demo)');
console.log(line);

// 1. Full report using the default export
console.log('1) Complete result report:');
console.log(buildReport(students));
console.log(line);

// 2. Topper using a named export
const topper = getTopper(students);
console.log(`2) Class topper: ${topper.name} (${topper.branch}) with average
${getAverage(topper)}`);

// 3. Filter by branch
const cseStudents = filterByBranch(students, 'CSE');
console.log(`3) CSE students (${cseStudents.length}): ${cseStudents.map(({ name }) =>
name).join(', ')}`);

// 4. Pass / fail summary
const passed = students.filter(hasPassed);
const failed = students.filter((student) => !hasPassed(student));
console.log(`4) Passed: ${passed.length} | Failed: ${failed.length} (pass marks =
${PASS_MARKS} in every subject)`);

// 5. Search by ID (found and not found)
const searchIds = [4, 10];
searchIds.forEach(id => {
  const student = findStudentById(students, id);
  if (student) {
    const average = getAverage(student);
    console.log(`5) Search ID ${id}: ${student.name}, average ${average}, grade
${getGrade(average)}`);
  } else {
    console.log(`5) Search ID ${id}: No student found with this ID.`);
  }
});

console.log(line);
console.log('All imported functions executed successfully.');
```

Expected Output

Run from inside the `question-2` folder:

```

$ node src/app.js
STUDENT RESULT MANAGEMENT (ES Modules Demo)
-----
1) Complete result report:
1  Aarav Sharma   CSE   Avg: 86.33  Grade: A   PASS
2  Diya Patel    CSE   Avg: 90.33  Grade: A+  PASS
3  Kabir Singh   ECE   Avg: 38.33  Grade: F   FAIL
4  Ananya Gupta  ME    Avg: 66.00  Grade: B   PASS
5  Rohan Verma   CSE   Avg: 58.33  Grade: C   PASS
-----
2) Class topper: Diya Patel (CSE) with average 90.33
3) CSE students (3): Aarav Sharma, Diya Patel, Rohan Verma
4) Passed: 4 | Failed: 1 (pass marks = 40 in every subject)
5) Search ID 4: Ananya Gupta, average 66, grade B
5) Search ID 10: No student found with this ID.
-----
All imported functions executed successfully.
```

(`npm start` gives exactly the same output.)

Every line of the output comes from a function that was imported from another module, which demonstrates that `export` and `import` are working correctly.

Explanation

Types of exports used

- **Named export** — `export const students = [...]` is imported with the same name inside braces: `import { students, PASS_MARKS } from './data/students.js'`.
- **Default export** — `export default buildReport` is imported without braces and can be given any name: `import buildReport, { getAverage } from './utils/studentUtils.js'`.
- In ES Modules the **file extension `.js` must be written** in relative imports.

Why modular JavaScript is better than one large file

1. **Separation of concerns** — each file has one job (data, logic or application flow), so the code is easier to understand.
2. **Reusability** — the functions in `studentUtils.js` can be imported by any other program (for example an API server) without copying code.
3. **Maintainability** — adding a student only changes `students.js`; changing the grading rule only changes `getGrade()`. A change in one place does not risk breaking unrelated code.
4. **Easier testing and debugging** — small pure functions can be tested individually, and errors are easier to locate.
5. **Encapsulation** — only exported values are visible to other files; everything else stays private to its module, avoiding global variable conflicts.
6. **Team work** — different developers can work on different modules at the same time with fewer merge conflicts.

Conclusion

The student application was successfully redesigned into a data module, a reusable logic module and a main application module using ES Modules. The program output confirms that both named and default imports work correctly. This modular structure makes the application cleaner, easier to extend, and allows the same logic to be reused in future backend projects.

3. Question 3 — Event Loop & Asynchronous Execution

Title: Demonstrating the Execution Order of Synchronous and Asynchronous Code in Node.js

Aim / Objective

To build a small Node.js program that mixes synchronous code with several asynchronous operations (timers, file reading, Promises and microtasks), record the actual order in which they run, and explain the role of the Call Stack, Node.js APIs, Event Loop and Callback Queue.

Approach

1. A helper function `log(type, message)` increases a step counter and stores every message in an `executionOrder` array, so the real execution order is recorded.
2. The script contains:
 - **Synchronous operations:** start message, a loop that adds numbers from 1 to 1,000,000, and an end message.
 - **Asynchronous operations:**
 - `setTimeout(..., 0)` — timer with zero delay
 - `fs.readFile()` — reading `sample.txt` from disk
 - `setTimeout(..., 100)` — timer with a longer delay
 - `Promise.resolve().then()` — Promise microtask
 - `queueMicrotask()` — microtask
3. The asynchronous operations are written **before** the heavy loop in the source code, to show that writing order is not the same as execution order.
4. When the last timer finishes, the program prints the final recorded order.

Files

File	Purpose
<code>app.js</code>	Event Loop demonstration program
<code>sample.txt</code>	Small text file read with <code>fs.readFile()</code>
<code>package.json</code>	Enables ES Modules (<code>"type": "module"</code>) and <code>npm start</code>

Code

`app.js`

```
// QUESTION 3 - EVENT LOOP & ASYNCHRONOUS EXECUTION
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A
//
// Every operation calls log() with its type. A step counter records the
// exact order in which Node.js actually ran each piece of code.
```

```

import { readFile } from 'node:fs';

let step = 0;
const executionOrder = [];

const log = (type, message) => {
  step += 1;
  executionOrder.push(`${step}. [${type}] ${message}`);
  console.log(`Step ${String(step).padStart(2)} | ${type.padEnd(10)} | ${message}`);
};

// Path is built from this file's location so it works from any folder.
const filePath = new URL('./sample.txt', import.meta.url);

console.log('EVENT LOOP DEMONSTRATION');
console.log('-'.repeat(70));

// ----- Synchronous code (runs on the Call Stack immediately) -----
log('SYNC', 'Program started');

// ----- Async 1: Timer with 0 ms delay (Timers phase / Callback Queue) -----
setTimeout(() => {
  log('TIMER', 'setTimeout with 0 ms delay finished');
}, 0);

// ----- Async 2: File read (handled by libuv thread pool, Poll phase) -----
readFile(filePath, 'utf8', (error, data) => {
  if (error) {
    log('FILE I/O', `Could not read file: ${error.message}`);
    return;
  }
  log('FILE I/O', `fs.readFile finished -> "${data.trim()}"`);
});

// ----- Async 3: Timer with a longer delay -----
setTimeout(() => {
  log('TIMER', 'setTimeout with 100 ms delay finished');
  printSummary();
}, 100);

// ----- Async 4: Promise (Microtask Queue) -----
Promise.resolve().then(() => {
  log('MICROTASK', 'Promise.then() callback executed');
});

// ----- Async 5: queueMicrotask (Microtask Queue) -----
queueMicrotask(() => {
  log('MICROTASK', 'queueMicrotask() callback executed');
});

// ----- More synchronous code -----
let sum = 0;
for (let i = 1; i <= 1_000_000; i += 1) {
  sum += i;
}
log('SYNC', `Heavy loop finished, sum of 1..1,000,000 = ${sum}`);
log('SYNC', 'Program finished (end of main script)');

function printSummary() {
  console.log('-'.repeat(70));
  console.log('FINAL EXECUTION ORDER:');
  executionOrder.forEach((entry) => console.log(`  ${entry}`));
}

```

sample.txt

```
Hello from sample.txt - this file was read asynchronously by fs.readFile().
```

Expected Output

Run from inside the `question-3` folder:

```
$ node app.js
EVENT LOOP DEMONSTRATION
-----
Step 1 | SYNC      | Program started
Step 2 | SYNC      | Heavy loop finished, sum of 1..1,000,000 = 500000500000
Step 3 | SYNC      | Program finished (end of main script)
Step 4 | MICROTASK  | Promise.then() callback executed
Step 5 | MICROTASK  | queueMicrotask() callback executed
Step 6 | TIMER      | setTimeout with 0 ms delay finished
Step 7 | FILE I/O   | fs.readFile finished -> "Hello from sample.txt - this file was read
asynchronously by fs.readFile()."
Step 8 | TIMER      | setTimeout with 100 ms delay finished
-----
FINAL EXECUTION ORDER:
  1. [SYNC] Program started
  2. [SYNC] Heavy loop finished, sum of 1..1,000,000 = 500000500000
  3. [SYNC] Program finished (end of main script)
  4. [MICROTASK] Promise.then() callback executed
  5. [MICROTASK] queueMicrotask() callback executed
  6. [TIMER] setTimeout with 0 ms delay finished
  7. [FILE I/O] fs.readFile finished -> "Hello from sample.txt - this file was read asynchronously by
fs.readFile()."
  8. [TIMER] setTimeout with 100 ms delay finished
```

Note: Steps 1–6 always appear in this order. Step 7 (file read) depends on disk speed; on almost every computer it finishes after the 0 ms timer and before the 100 ms timer, as shown.

Explanation

Main components

Component	Role in this program
Call Stack	The place where JavaScript actually runs functions, one at a time (Node.js runs JavaScript on a single main thread). The whole main script, including the heavy loop, runs on the Call Stack first. Nothing else can run until the stack is empty.
Node.js APIs (libuv)	Features provided by Node.js outside the JavaScript engine, such as timers (<code>setTimeout</code>) and file system access (<code>fs.readFile</code>). When we call them, Node.js starts the work in the background (timer system or libuv thread pool) and JavaScript continues with the next line immediately.
Callback Queue (Task / Macrotask Queue)	When a background operation finishes (timer expires, file is read), its callback is placed in a queue to wait. In Node.js there are separate queues for each Event Loop phase — e.g. the timers queue and the poll (I/O) queue.

Component	Role in this program
Microtask Queue	A higher-priority queue for <code>Promise.then()</code> and <code>queueMicrotask()</code> callbacks. It is emptied completely as soon as the Call Stack becomes empty, before the Event Loop moves to any callback queue.
Event Loop	A continuous loop that checks: “Is the Call Stack empty?” If yes, it first runs all microtasks, then takes the next ready callback from the callback queues (timers → I/O poll → check ...) and pushes it onto the Call Stack.

Step-by-step execution sequence

1. **Step 1 - SYNC:** `log('SYNC', 'Program started')` runs directly on the Call Stack.
2. `setTimeout(..., 0)` is handed to the **Node.js timer API**. The callback is **not** run now.
3. `readFile()` is handed to **libuv**, which reads the file in a background thread.
4. `setTimeout(..., 100)` is registered with the timer API.
5. `Promise.resolve().then(...)` and `queueMicrotask(...)` put their callbacks into the **Microtask Queue**.
6. **Step 2 - SYNC:** the heavy loop runs completely on the Call Stack (it blocks everything else while it runs).
7. **Step 3 - SYNC:** “Program finished” is printed. The main script ends and the **Call Stack becomes empty**.
8. **Steps 4-5 - MICROTASKS:** the Event Loop first empties the Microtask Queue, in the order the callbacks were added: `Promise.then()` and then `queueMicrotask()`.
9. **Step 6 - TIMER (0 ms):** the Event Loop enters the **timers phase**. The 0 ms timer has already expired (the loop took longer than that), so its callback moves from the callback queue to the Call Stack.
10. **Step 7 - FILE I/O:** in the **poll phase**, the completed file read is found and its callback runs, printing the file content.
11. **Step 8 - TIMER (100 ms):** after about 100 ms, the second timer callback runs in the next timers phase and prints the final summary. No work is left, so the process exits.

Why can an asynchronous callback with a very small delay run after synchronous code?

- `setTimeout(fn, 0)` does **not** mean “run immediately”. It means “run `fn` **no earlier than** 0 ms from now”. (Node.js actually treats 0 as a minimum of 1 ms.)
- When the timer expires, the callback is only **placed in the Callback Queue**. It still has to wait for the Event Loop to move it to the Call Stack.
- The Event Loop only takes a callback from the queue **when the Call Stack is empty**. While synchronous code (like our 1,000,000-iteration loop) is running, the stack is busy, so the callback must wait — even though its delay finished long ago.
- In addition, all **microtasks** (Promises, `queueMicrotask`) are run before the next callback is taken from the timers queue.
- Therefore, the delay is only the **minimum** waiting time, not a guaranteed execution time. This is why Steps 1–5 appear before the 0 ms timer.

Conclusion

The program clearly shows that Node.js runs all synchronous code first on the Call Stack, while asynchronous operations are handled by Node.js APIs in the background. When the stack is empty, the Event Loop runs microtasks first and then callbacks from the callback queues (timers, then I/O). Understanding this order helps backend developers avoid blocking the main thread and predict when callbacks will execute.

4. Question 4 — Reliable Asynchronous Application

Title: Asynchronous Student Marks Processor with Error Handling

Aim / Objective

To build a small asynchronous Node.js program that retrieves data from a JSON file and processes it using Promises and `async/await`, and handles both successful and failed operations using `try...catch...finally`, meaningful success messages and user-friendly error messages.

Approach

1. **Retrieve data:** `loadRecords()` reads a JSON file using `fs/promises` (`readFile` returns a Promise) and parses it.
2. **Process data:** `processRecords()` returns a `new Promise(...)` that, after a short delay, either
 - **resolves** with a summary (count, average, topper, passed), or
 - **rejects** with a custom `ValidationError` if any marks are invalid.
3. **Control flow:** `runCase()` is an `async` function that uses `await` inside `try...catch...finally`:
 - `try` → runs the steps and prints a **SUCCESS** message.
 - `catch` → converts any error into a **friendly ERROR message** using `getFriendlyMessage()`.
 - `finally` → always stops the progress-indicator timer (cleanup) and prints the time taken.
4. Three test cases are provided:
 - `valid` → correct data → **success**
 - `invalid` → wrong marks (`"ninety"`) → **validation failure**
 - `missing` → file does not exist → **file failure**

Files

File	Purpose
<code>app.js</code>	The asynchronous processing program
<code>data/valid-marks.json</code>	Correct input data (success case)
<code>data/invalid-marks.json</code>	Input with invalid marks (failure case)
<code>package.json</code>	Enables ES Modules and adds test scripts

Code

app.js

```
// QUESTION 4 - RELIABLE ASYNCHRONOUS APPLICATION
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A
//
// The program reads a JSON file of student marks asynchronously,
// validates it, and calculates a small result summary.
//
// Usage:
//   node app.js valid      -> success case (data/valid-marks.json)
//   node app.js invalid    -> failure case: bad data inside the file
//   node app.js missing    -> failure case: file does not exist
//   node app.js            -> runs all three cases one after another

import { readFile } from 'node:fs/promises';

// Custom error class so we can tell "expected" validation errors
// apart from unexpected programming errors.
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ValidationError';
  }
}

const DATA_FILES = {
  valid: 'valid-marks.json',
  invalid: 'invalid-marks.json',
  missing: 'does-not-exist.json',
};

// Promise-based helper: simulates a slow operation (like a network/database call).
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// Step 1: Retrieve data asynchronously (fs/promises returns a Promise).
const loadRecords = async (fileName) => {
  const fileUrl = new URL(`./data/${fileName}`, import.meta.url);
  const text = await readFile(fileUrl, 'utf8');
  return JSON.parse(text);
};

// Step 2: Process data - returns a Promise that resolves or rejects.
const processRecords = (records) =>
  new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!Array.isArray(records) || records.length === 0) {
        reject(new ValidationError('The file must contain a non-empty list of students.'));
        return;
      }

      const badRecord = records.find(
        ({ marks }) => typeof marks !== 'number' || marks < 0 || marks > 100,
      );
      if (badRecord) {
        reject(new ValidationError(
          `Invalid marks "${badRecord.marks}" for ${badRecord.name}. Marks must be a number
between 0 and 100.`
        ));
        return;
      }

      const total = records.reduce((sum, { marks }) => sum + marks, 0);
      const topper = records.reduce((best, cur) => (cur.marks > best.marks ? cur : best));
    }, 1000);
  });
}
```

```

        resolve({
            count: records.length,
            average: (total / records.length).toFixed(2),
            topper: topper.name,
            passed: records.filter(({ marks }) => marks >= 40).length,
        });
    }, 300);
});

// Converts any technical error into a message that a normal user can understand.
const getFriendlyMessage = (error) => {
    if (error instanceof ValidationError) return error.message;
    if (error.code === 'ENOENT') return 'The data file was not found. Please check the file
name and try again.';
    if (error instanceof SyntaxError) return 'The data file is not valid JSON.';
    return 'Something unexpected went wrong. Please try again later.';
};

// Main async workflow with try...catch...finally
const runCase = async (caseName) => {
    const fileName = DATA_FILES[caseName];
    console.log(`\n=== Case: ${caseName.toUpperCase()} (${fileName}) ===`);

    // A simple progress indicator that must always be stopped (cleanup).
    const progress = setInterval(() => process.stdout.write('.'), 100);
    const startTime = Date.now();
    process.stdout.write('Processing');

    try {
        const records = await loadRecords(fileName);
        await delay(200);
        const { count, average, topper, passed } = await processRecords(records);

        process.stdout.write('\n');
        console.log(`SUCCESS: Processed ${count} student records.`);
        console.log(`          Class average: ${average} | Topper: ${topper} | Passed:
${passed}/${count}`);
        return true;
    } catch (error) {
        process.stdout.write('\n');
        console.log(`ERROR: ${getFriendlyMessage(error)}`);
        // Technical code is kept short for developers; the user sees the friendly text above.
        console.log(`          (error type: ${error.code ?? error.name})`);
        return false;
    } finally {
        // finally runs in BOTH success and failure, so the timer never leaks.
        clearInterval(progress);
        console.log(`CLEANUP: Progress timer stopped. Time taken: ${Date.now() - startTime} ms`);
    }
};

const main = async () => {
    const [requested] = process.argv.slice(2);

    if (requested && !DATA_FILES[requested]) {
        console.log(`ERROR: Unknown case "${requested}". Use one of:
${Object.keys(DATA_FILES).join(', ')}`);
        process.exitCode = 1;
        return;
    }

    const cases = requested ? [requested] : Object.keys(DATA_FILES);
    for (const caseName of cases) {
        await runCase(caseName);
    }
    console.log('\nApplication finished normally - no crash occurred.');
```

```
main();
```

data/valid-marks.json

```
[
  { "name": "Aarav Sharma", "marks": 86 },
  { "name": "Diya Patel", "marks": 91 },
  { "name": "Kabir Singh", "marks": 38 },
  { "name": "Ananya Gupta", "marks": 66 }
]
```

data/invalid-marks.json

```
[
  { "name": "Rohan Verma", "marks": 72 },
  { "name": "Meera Iyer", "marks": "ninety" },
  { "name": "Ishaan Rao", "marks": 105 }
]
```

package.json

```
{
  "name": "question-4-reliable-async",
  "version": "1.0.0",
  "description": "Module 2 - Question 4: Reliable asynchronous data processing with error handling",
  "type": "module",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "test:success": "node app.js valid",
    "test:invalid": "node app.js invalid",
    "test:missing": "node app.js missing"
  },
  "author": "SAGAR KUMAR (SOE25BTAM29)",
  "license": "ISC"
}
```

Expected Output

Run the commands from inside the `question-4` folder. (The number of dots and the time in ms may differ slightly on your computer.)

Test 1 — Successful case

```
$ node app.js valid

=== Case: VALID (valid-marks.json) ===
Processing....
SUCCESS: Processed 4 student records.
      Class average: 70.25 | Topper: Diya Patel | Passed: 3/4
CLEANUP: Progress timer stopped. Time taken: 504 ms

Application finished normally - no crash occurred.
```

Test 2 — Failure case (invalid data)

```
$ node app.js invalid

=== Case: INVALID (invalid-marks.json) ===
Processing.....
ERROR: Invalid marks "ninety" for Meera Iyer. Marks must be a number between 0 and 100.
      (error type: ValidationError)
CLEANUP: Progress timer stopped. Time taken: 504 ms

Application finished normally - no crash occurred.
```

Test 3 — Failure case (missing file)

```
$ node app.js missing

=== Case: MISSING (does-not-exist.json) ===
Processing
ERROR: The data file was not found. Please check the file name and try again.
      (error type: ENOENT)
CLEANUP: Progress timer stopped. Time taken: 1 ms

Application finished normally - no crash occurred.
```

Run all cases together

```
$ node app.js

=== Case: VALID (valid-marks.json) ===
Processing.....
SUCCESS: Processed 4 student records.
       Class average: 70.25 | Topper: Diya Patel | Passed: 3/4
CLEANUP: Progress timer stopped. Time taken: 505 ms

=== Case: INVALID (invalid-marks.json) ===
Processing...
ERROR: Invalid marks "ninety" for Meera Iyer. Marks must be a number between 0 and 100.
      (error type: ValidationError)
CLEANUP: Progress timer stopped. Time taken: 503 ms

=== Case: MISSING (does-not-exist.json) ===
Processing
ERROR: The data file was not found. Please check the file name and try again.
      (error type: ENOENT)
CLEANUP: Progress timer stopped. Time taken: 1 ms

Application finished normally - no crash occurred.
```

The same tests can also be run with `npm run test:success`, `npm run test:invalid` and `npm run test:missing`.

Explanation

Requirement	Implementation
Promise-based execution	<code>readFile</code> from <code>fs/promises</code> , <code>delay()</code> and <code>processRecords()</code> all return Promises (<code>new Promise((resolve, reject) => ...)</code>).

Requirement	Implementation
<code>async/await</code>	<code>loadRecords</code> , <code>runCase</code> and <code>main</code> are <code>async</code> functions; <code>await</code> waits for each Promise in a readable top-to-bottom style.
<code>try...catch</code>	Every awaited step in <code>runCase</code> is inside one <code>try</code> block; a rejected Promise automatically jumps to <code>catch</code> .
Appropriate error handling	A custom <code>ValidationError</code> class separates bad input from system errors such as <code>ENOENT</code> (file not found) and <code>SyntaxError</code> (bad JSON).
Meaningful success message	<code>SUCCESS: Processed 4 student records.</code> followed by average, topper and pass count.
User-friendly error message	<code>getFriendlyMessage()</code> turns technical errors into plain sentences, e.g. <i>"The data file was not found. Please check the file name and try again."</i>
<code>finally</code> for cleanup	The progress <code>setInterval</code> is cleared in <code>finally</code> , so it is stopped in both success and failure. Without this, the timer would keep the process running forever.

How this approach prevents unexpected application failure

1. **No unhandled Promise rejections** — every `await` is inside `try...catch`, so a rejected Promise is caught instead of crashing Node.js with an `UnhandledPromiseRejection` error.
2. **Input is validated before use** — wrong data types or out-of-range marks are rejected early with a clear reason instead of producing a wrong average (`NaN`).
3. **Different errors are handled differently** — validation errors, missing files and invalid JSON each get a specific message, and any unknown error still gets a safe generic message.
4. **Resources are always released** — `finally` guarantees the timer is cleared, so the program exits normally in every case.
5. **One failure does not stop the rest** — when all three cases run together, the failure of case 2 does not prevent case 3 from running, and the program ends with *"Application finished normally - no crash occurred."*

Conclusion

The program demonstrates reliable asynchronous programming with Promises and `async/await`. Using `try...catch` for errors, a custom error class for validation, user-friendly messages and `finally` for cleanup, the application handles both success and failure gracefully and never crashes unexpectedly.

5. Question 5 — Mini Project: Modular Backend Utility

1. Project Title

Task Manager — A Command-Line Task Management Utility built with Node.js, ES Modules, `fs/promises` and `.env` configuration

2. Student Details

Field	Details
Name	SAGAR KUMAR
PRN	SOE25BTAM29
Section	A
Module	Module 2 — Modern JavaScript for Backend Development
Question	Question 5 — Mini Project: Modular Backend Utility

3. Project Overview

Task Manager is a small command-line (CLI) backend utility that helps a user manage daily tasks. A user can add a task, view all tasks, mark a task as completed and delete a task directly from the terminal. Tasks are stored in a JSON file (`data/tasks.json`), so no database is required. All file operations are asynchronous, the code is organised into ES Modules, and the application name and data-file location are read from a `.env` file instead of being hardcoded.

4. Objectives

- Organise a Node.js application into separate ES Modules (configuration, data, services, main app).
- Use modern JavaScript features: `let / const` , arrow functions, template literals, destructuring, spread operator, default values and optional chaining.
- Perform asynchronous file operations using Promises and `async/await` (`fs/promises`).
- Validate user input and handle errors with clear, friendly messages.
- Store configuration in `.env` and read it using `process.env` (via `dotenv`).
- Demonstrate both successful execution and error scenarios.

5. Technologies Used

Technology	Purpose
Node.js (v18 or above, LTS recommended)	JavaScript runtime

Technology	Purpose
JavaScript (ES6+, ES Modules)	Programming language and module system
<code>node:fs/promises</code> (built-in)	Asynchronous reading/writing of the JSON file
<code>node:path</code> , <code>node:url</code> (built-in)	Building file paths that work on Windows, macOS and Linux
<code>dotenv</code> (only external dependency)	Loads variables from <code>.env</code> into <code>process.env</code>
JSON file	Simple persistent storage for tasks

6. Features

- **Add task** — creates a task with an auto-incremented ID, `completed: false` and a `createdAt` timestamp.
- **List tasks** — shows all tasks in a table with status and a summary (total / completed / pending).
- **Complete task** — marks a task as completed and stores `completedAt`.
- **Delete task** — removes a task by ID.
- **Input validation** — empty title, title longer than 100 characters, duplicate title, non-numeric or negative ID.
- **Friendly error handling** — invalid task ID, task not found, already completed task, unknown command, missing `.env` configuration and corrupted data file.
- **Help command** — lists all available commands.
- **Automatic data file creation** — if `tasks.json` or the `data` folder is missing, it is created automatically on the first add.

7. Folder Structure

```
question-5-task-manager/
├── src/
│   ├── config/
│   │   └── config.js          # Loads and validates .env configuration
│   ├── data/
│   │   └── taskRepository.js  # Reads/writes tasks.json (async fs/promises)
│   ├── services/
│   │   └── taskService.js     # Business logic + validation + TaskError
│   └── app.js                 # Entry point: parses CLI command and prints results
├── data/
│   └── tasks.json             # Task storage (starts as an empty array [])
├── .env.example               # Sample environment configuration
├── .gitignore                 # Ignores node_modules/ and .env
├── package.json               # "type": "module", scripts and dotenv dependency
├── package-lock.json          # Exact dependency versions (created by npm install)
└── README.md                  # Project documentation (this file)
```

File	Responsibility
<code>src/config/config.js</code>	Application configuration module. Loads <code>.env</code> with <code>dotenv</code> , checks that <code>APP_NAME</code> and <code>DATA_FILE</code> exist, converts <code>DATA_FILE</code> into an absolute path and exports a frozen <code>config</code> object.

File	Responsibility
src/data/taskRepository.js	Data module. The only file that touches the disk. Exports <code>readTasks()</code> and <code>saveTasks()</code> using <code>fs/promises</code> .
src/services/taskService.js	Logic module. Exports <code>addTask</code> , <code>listTasks</code> , <code>completeTask</code> , <code>deleteTask</code> and the custom <code>TaskError</code> class. Validates input and applies business rules.
src/app.js	Main application module. Reads the command from <code>process.argv</code> , calls the correct service function and prints success or error messages.

8. Installation Steps

Prerequisite: Node.js 18 or newer (check with `node -v`).

```
# 1. Open a terminal inside the project folder
cd module-2-backend-assignment/question-5-task-manager

# 2. Install the only dependency (dotenv)
npm install
```

9. Environment Setup

Create a `.env` file by copying the example file:

```
# macOS / Linux / Git Bash
cp .env.example .env

# Windows Command Prompt
copy .env.example .env

# Windows PowerShell
Copy-Item .env.example .env
```

`.env.example` / `.env` content:

```
APP_NAME=Task Manager
DATA_FILE=./data/tasks.json
```

Variable	Meaning	Used in
APP_NAME	Name printed in the header of every command	src/config/config.js → <code>config.appName</code>
DATA_FILE	Path of the JSON file where tasks are stored (relative to the project folder)	src/config/config.js → <code>config.dataFile</code>

`.env` is listed in `.gitignore`, so real configuration is never committed — only `.env.example` is shared.

10. How to Run

```
node src/app.js <command> [arguments]
```

or using the npm script (note the `--` before the arguments):

```
npm start -- <command> [arguments]
```

`npm run dev` starts the app in watch mode (`node --watch`), which restarts it automatically when a source file changes — useful while developing.

Tip: Before taking screenshots, make sure `data/tasks.json` contains only `[]` so that task IDs start from 1.

11. Available Commands

Command	Description	Example
<code>add "<title>"</code>	Add a new task	<code>node src/app.js add "Complete backend assignment"</code>
<code>list</code>	Show all tasks	<code>node src/app.js list</code>
<code>complete <id></code>	Mark a task as completed	<code>node src/app.js complete 1</code>
<code>delete <id></code>	Delete a task	<code>node src/app.js delete 1</code>
<code>help</code>	Show the help message	<code>node src/app.js help</code>

12. Example Successful Execution

The following output was produced by running the commands one after another starting with an empty `data/tasks.json`. (Dates and times will show your own system time.)

```
$ node src/app.js help
=== Task Manager ===

Task Manager - available commands:
node src/app.js add "<task title>"  Add a new task
node src/app.js list                Show all tasks
node src/app.js complete <id>      Mark a task as completed
node src/app.js delete <id>        Delete a task
node src/app.js help                Show this help message

$ node src/app.js add "Complete backend assignment"
=== Task Manager ===
Task added successfully: "Complete backend assignment" (ID: 1)

$ node src/app.js add "Prepare for DBMS quiz"
=== Task Manager ===
Task added successfully: "Prepare for DBMS quiz" (ID: 2)

$ node src/app.js add "Submit Module 2 report"
=== Task Manager ===
Task added successfully: "Submit Module 2 report" (ID: 3)
```

```
$ node src/app.js list
=== Task Manager ===
ID   STATUS   TITLE                                     CREATED
-----
1    [ ]      Complete backend assignment             10/6/2026, 9:43:45 AM
2    [ ]      Prepare for DBMS quiz                  10/6/2026, 9:43:45 AM
3    [ ]      Submit Module 2 report                 10/6/2026, 9:43:45 AM
-----
Total: 3 | Completed: 0 | Pending: 3

$ node src/app.js complete 1
=== Task Manager ===
Task 1 marked as completed: "Complete backend assignment"

$ node src/app.js delete 2
=== Task Manager ===
Task deleted successfully: "Prepare for DBMS quiz" (ID: 2)

$ node src/app.js list
=== Task Manager ===
ID   STATUS   TITLE                                     CREATED
-----
1    [done]   Complete backend assignment             10/6/2026, 9:43:45 AM
3    [ ]      Submit Module 2 report                 10/6/2026, 9:43:45 AM
-----
Total: 2 | Completed: 1 | Pending: 1
```

Contents of `data/tasks.json` after these commands:

```
[
  {
    "id": 1,
    "title": "Complete backend assignment",
    "completed": true,
    "createdAt": "2026-10-06T04:13:45.170Z",
    "completedAt": "2026-10-06T04:13:45.422Z"
  },
  {
    "id": 3,
    "title": "Submit Module 2 report",
    "completed": false,
    "createdAt": "2026-10-06T04:13:45.303Z"
  }
]
```

13. Example Error Handling

Continuing from the state above:

```
$ node src/app.js complete 99
=== Task Manager ===
ERROR: Task with ID 99 was not found.

$ node src/app.js delete 99
=== Task Manager ===
ERROR: Task with ID 99 was not found.

$ node src/app.js complete abc
=== Task Manager ===
ERROR: Invalid task ID "abc". The ID must be a positive whole number.

$ node src/app.js add ""
=== Task Manager ===
```

```
ERROR: Task title cannot be empty. Usage: node src/app.js add "Your task title"
```

```
$ node src/app.js add "Complete backend assignment"
=== Task Manager ===
ERROR: A task with the title "Complete backend assignment" already exists.
```

```
$ node src/app.js complete 1
=== Task Manager ===
ERROR: Task with ID 1 is already completed.
```

```
$ node src/app.js remove 1
=== Task Manager ===
ERROR: Unknown command "remove".
```

```
Task Manager - available commands:
node src/app.js add "<task title>"  Add a new task
node src/app.js list                 Show all tasks
node src/app.js complete <id>       Mark a task as completed
node src/app.js delete <id>         Delete a task
node src/app.js help                 Show this help message
```

Missing configuration (when `.env` has not been created):

```
$ node src/app.js list
CONFIG ERROR: Missing environment variable(s): APP_NAME, DATA_FILE
Create a .env file by copying .env.example (see README.md), then run the command again.
```

Corrupted data file (if `tasks.json` contains invalid JSON, e.g. `{bad json}`):

```
$ node src/app.js list
=== Task Manager ===
ERROR: Something went wrong - The data file "tasks.json" is corrupted (invalid JSON). Fix it or
replace its content with [].
```

In every error case the program prints a clear message and exits with exit code `1` instead of crashing with a stack trace.

14. Explanation of Asynchronous Operations

- `taskRepository.js` uses `readFile`, `writeFile` and `mkdir` from `node:fs/promises`. Each of these returns a **Promise**, so the file is read/written without blocking the main thread.
- Every service function (`addTask`, `completeTask`, ...) is an **async function** and uses `await` to wait for the file operation to finish before continuing. For example:

```
const tasks = await readTasks();    // wait for file to be read
await saveTasks([...tasks, newTask]); // wait for file to be written
```

- In `app.js`, each command handler is `async` and `main()` uses `await handler(args)` inside `try...catch`, so any rejected Promise is caught and turned into a friendly message.
- Using `async/await` keeps the code readable from top to bottom, like synchronous code, while still being non-blocking.

15. Explanation of ES Modules

- `package.json` contains `"type": "module"`, so Node.js treats all `.js` files as **ES Modules**.
- Modules share code with **export** and **import** (no `require()` is used anywhere):
 - `config.js` → `export default config`
 - `taskRepository.js` → `export const readTasks`, `export const saveTasks`
 - `taskService.js` → named exports `addTask`, `listTasks`, `completeTask`, `deleteTask`, `TaskError`
 - `app.js` → `import config from './config/config.js'` and `import { addTask, ... } from './services/taskService.js'`
- Built-in modules are imported with the `node:` prefix (e.g. `import { readFile } from 'node:fs/promises'`).
- Because ES Modules have no `__dirname`, `config.js` uses `import.meta.url` with `fileURLToPath()` to find the project folder. This lets the app find `.env` and `tasks.json` correctly even when it is started from another folder.
- Dependency flow is one-way and easy to follow: `app.js` → `taskService.js` → `taskRepository.js` → `config.js`.

16. Error Handling Approach

1. **Custom error class** — `TaskError` (in `taskService.js`) represents expected errors caused by user input (empty title, invalid ID, task not found, duplicate title, already completed).
2. **Validation before any file change** — `validateTitle()` and `parseId()` check input first, so invalid data is never written to `tasks.json`.
3. **Central try...catch in app.js** — all command handlers are awaited inside one `try...catch`.
 - `TaskError` → prints only the friendly message.
 - Any other error (e.g. permission denied, corrupted file) → prints `Something went wrong - ...` instead of an ugly stack trace.
4. **Safe file reading** — `readTasks()` returns an empty list when the file does not exist (`ENOENT`) and gives a clear message when the JSON is corrupted.
5. **Configuration check at startup** — `config.js` stops the program with a clear message if `.env` values are missing.
6. **Exit codes** — `process.exitCode = 1` is set on errors, so scripts and other tools can detect that the command failed, while the process still ends normally.

17. Conclusion

The Task Manager mini project fulfils all the requirements of Question 5. It is organised into ES Modules with separate configuration, data and logic layers; it uses modern JavaScript features such as arrow functions, template literals, destructuring and the spread operator; all file operations are asynchronous using `fs/promises` with `async/await`; configuration is loaded from `.env` through `process.env`; and both successful operations and error scenarios are handled with clear messages. The project is simple, but its structure can easily be extended — for example, by replacing the JSON file with a database or adding a REST API — without changing the business logic.

Appendix — Complete Source Code

package.json

```
{
  "name": "task-manager",
  "version": "1.0.0",
  "description": "Module 2 - Question 5: CLI Task Manager built with Node.js ES Modules,
  async fs/promises and dotenv",
  "type": "module",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js",
    "dev": "node --watch src/app.js"
  },
  "keywords": ["nodejs", "es-modules", "cli", "task-manager"],
  "author": "SAGAR KUMAR (SOE25BTAM29)",
  "license": "ISC",
  "engines": {
    "node": ">=18"
  },
  "dependencies": {
    "dotenv": "^16.4.7"
  }
}
```

src/config/config.js

```
// Configuration module: loads settings from the .env file using dotenv.
// No configuration value is hardcoded anywhere else in the application.
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A

import dotenv from 'dotenv';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

// Project root = two folders above this file (src/config -> project root).
// This lets the app work even if it is started from another folder.
const projectRoot = path.resolve(path.dirname(fileURLToPath(import.meta.url)), '..', '..');

dotenv.config({ path: path.join(projectRoot, '.env') });

const REQUIRED_VARIABLES = ['APP_NAME', 'DATA_FILE'];

const missing = REQUIRED_VARIABLES.filter((name) => !process.env[name]?.trim());

// Stop early with a clear message instead of failing later with a confusing error.
if (missing.length > 0) {
  console.error(`CONFIG ERROR: Missing environment variable(s): ${missing.join(', ')}');
  console.error('Create a .env file by copying .env.example (see README.md), then run the
command again.');
```

```
  process.exit(1);
}

const { APP_NAME, DATA_FILE } = process.env;

const config = Object.freeze({
  appName: APP_NAME.trim(),
  // Relative paths in .env are resolved from the project root.
  dataFile: path.resolve(projectRoot, DATA_FILE.trim()),
});
```

```
export default config;
```

src/data/taskRepository.js

```
// Data module (repository): the ONLY place that reads/writes the JSON file.
// All file operations are asynchronous and use fs/promises.
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A

import { readFile, writeFile, mkdir } from 'node:fs/promises';
import path from 'node:path';
import config from '../config/config.js';

const { dataFile } = config;

// Reads all tasks. If the file does not exist yet, start with an empty list.
export const readTasks = async () => {
  try {
    const content = await readFile(dataFile, 'utf8');
    if (!content.trim()) return [];

    const tasks = JSON.parse(content);
    if (!Array.isArray(tasks)) {
      throw new SyntaxError('Data file must contain a JSON array.');
```

src/services/taskService.js

```
// Service module (business logic): validation and task operations.
// It uses the repository for storage, so it never touches files directly.
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A

import { readTasks, saveTasks } from '../data/taskRepository.js';

const MAX_TITLE_LENGTH = 100;

// Custom error for problems caused by user input (expected errors).
export class TaskError extends Error {
  constructor(message) {
    super(message);
    this.name = 'TaskError';
  }
}

// ----- Validation helpers -----
const validateTitle = (title) => {
  const cleanTitle = String(title ?? '').trim();
  if (!cleanTitle) {
```

```

        throw new TaskError('Task title cannot be empty. Usage: node src/app.js add "Your task title"');
    }
    if (cleanTitle.length > MAX_TITLE_LENGTH) {
        throw new TaskError(`Task title is too long (${cleanTitle.length} characters). Maximum allowed is ${MAX_TITLE_LENGTH}.`);
    }
    return cleanTitle;
};

const parseId = (value) => {
    const id = Number(value);
    if (value === undefined || !Number.isInteger(id) || id <= 0) {
        throw new TaskError(`Invalid task ID "${value ?? ''}". The ID must be a positive whole number.`);
    }
    return id;
};

const findTaskIndex = (tasks, id) => {
    const index = tasks.findIndex((task) => task.id === id);
    if (index === -1) {
        throw new TaskError(`Task with ID ${id} was not found.`);
    }
    return index;
};

// ----- Task operations (all async) -----
export const addTask = async (title) => {
    const cleanTitle = validateTitle(title);
    const tasks = await readTasks();

    const duplicate = tasks.some((task) => task.title.toLowerCase() === cleanTitle.toLowerCase());
    if (duplicate) {
        throw new TaskError(`A task with the title "${cleanTitle}" already exists.`);
    }

    const nextId = tasks.length > 0 ? Math.max(...tasks.map(({ id }) => id)) + 1 : 1;
    const newTask = {
        id: nextId,
        title: cleanTitle,
        completed: false,
        createdAt: new Date().toISOString(),
    };

    await saveTasks([...tasks, newTask]);
    return newTask;
};

export const listTasks = async () => readTasks();

export const completeTask = async (idValue) => {
    const id = parseId(idValue);
    const tasks = await readTasks();
    const index = findTaskIndex(tasks, id);

    if (tasks[index].completed) {
        throw new TaskError(`Task with ID ${id} is already completed.`);
    }

    tasks[index] = { ...tasks[index], completed: true, completedAt: new Date().toISOString() };
    await saveTasks(tasks);
    return tasks[index];
};

export const deleteTask = async (idValue) => {
    const id = parseId(idValue);

```

```

const tasks = await readTasks();
const index = findTaskIndex(tasks, id);

const [deletedTask] = tasks.splice(index, 1);
await saveTasks(tasks);
return deletedTask;
};

```

src/app.js

```

// Main application module: reads the command from the terminal,
// calls the service layer and prints friendly messages.
// Student: SAGAR KUMAR | PRN: SOE25BTAM29 | Section: A
//
// Usage:
//   node src/app.js add "Complete backend assignment"
//   node src/app.js list
//   node src/app.js complete 1
//   node src/app.js delete 1
//   node src/app.js help

import config from './config/config.js';
import { addTask, listTasks, completeTask, deleteTask, TaskError } from
'./services/taskService.js';

const { appName } = config;

const showHelp = () => {
  console.log(`
${appName} - available commands:
node src/app.js add "<task title>"    Add a new task
node src/app.js list                  Show all tasks
node src/app.js complete <id>         Mark a task as completed
node src/app.js delete <id>          Delete a task
node src/app.js help                  Show this help message
`);
};

const printTasks = (tasks) => {
  if (tasks.length === 0) {
    console.log('No tasks found. Add one with: node src/app.js add "Your task"');
    return;
  }

  console.log('ID    STATUS    TITLE                                     CREATED');
  console.log('-'.repeat(78));
  tasks.forEach(({ id, title, completed, createdAt }) => {
    const status = completed ? '[done]' : '[ ]';
    const created = new Date(createdAt).toLocaleString();
    console.log(`${String(id).padEnd(4)} ${status.padEnd(9)} ${title.padEnd(41)}
${created}`);
  });

  const doneCount = tasks.filter(({ completed }) => completed).length;
  console.log('-'.repeat(78));
  console.log(`Total: ${tasks.length} | Completed: ${doneCount} | Pending: ${tasks.length -
doneCount}`);
};

// Command handlers stored in an object instead of a long if/else chain.
const commands = {
  add: async (args) => {
    const task = await addTask(args.join(' '));
    console.log(`Task added successfully: "${task.title}" (ID: ${task.id})`);
  },
  list: async () => {

```

```

    printTasks(await listTasks());
  },
  complete: async ([id]) => {
    const task = await completeTask(id);
    console.log(`Task ${task.id} marked as completed: "${task.title}"`);
  },
  delete: async ([id]) => {
    const task = await deleteTask(id);
    console.log(`Task deleted successfully: "${task.title}" (ID: ${task.id})`);
  },
  help: async () => showHelp(),
};

const main = async () => {
  const [command = 'help', ...args] = process.argv.slice(2);

  console.log(`=== ${appName} ===`);

  const handler = commands[command.toLowerCase()];
  if (!handler) {
    console.error(`ERROR: Unknown command "${command}".`);
    showHelp();
    process.exitCode = 1;
    return;
  }

  try {
    await handler(args);
  } catch (error) {
    if (error instanceof TaskError) {
      // Expected error caused by user input -> friendly message only.
      console.error(`ERROR: ${error.message}`);
    } else {
      // Unexpected error (file permission, corrupted file, etc.).
      console.error(`ERROR: Something went wrong - ${error.message}`);
    }
    process.exitCode = 1;
  }
};

main();

```

.gitignore

```

node_modules/
.env
!.env.example

```

data/tasks.json (initial content)

```
[ ]
```

Screenshot Checklist (Question 5)

Run these commands inside `question-5-task-manager` (with `data/tasks.json` set to `[ ]` first) and take one screenshot of each terminal screen:

#	What to capture	Command(s)
1	Project installed successfully	<code>npm install</code>
2	Help / available commands	<code>node src/app.js help</code>
3	Adding tasks (success)	<code>node src/app.js add "Complete backend assignment"</code> then <code>node src/app.js add "Prepare for DBMS quiz"</code> then <code>node src/app.js add "Submit Module 2 report"</code>
4	Listing tasks	<code>node src/app.js list</code>
5	Completing and deleting a task	<code>node src/app.js complete 1</code> then <code>node src/app.js delete 2</code> then <code>node src/app.js list</code>
6	Error: task not found	<code>node src/app.js complete 99</code> and <code>node src/app.js delete 99</code>
7	Error: invalid input	<code>node src/app.js complete abc</code> and <code>node src/app.js add ""</code>
8	Error: missing <code>.env</code> configuration (optional)	Temporarily rename <code>.env</code> to <code>.env.backup</code> , run <code>node src/app.js list</code> , then rename it back
9	Folder structure in VS Code / file explorer	Expand all folders in the sidebar
10	<code>.env</code> usage	Open <code>.env.example</code> and <code>src/config/config.js</code> side by side

6. Testing & Execution

6.1 Installation

```
# Check Node.js version (18 or above required)
node -v

# Only Question 5 has an external dependency
cd module-2-backend-assignment/question-5-task-manager
npm install
cp .env.example .env      # Windows CMD: copy .env.example .env
```

6.2 Test Summary

#	Question	Test	Command	Type	Result
1	Q1	Legacy code runs	<code>node legacy.js</code>	Success	Billing report printed
2	Q1	Modern code gives same totals	<code>node app.js</code>	Success	Same totals + safe handling of order 104
3	Q1	Reassigning a <code>const</code> / frozen object	<code>node app.js</code>	Error (handled)	<code>TypeError</code> caught and printed
4	Q2	Imports from data and utils modules	<code>node src/app.js</code>	Success	Report, topper, filters printed
5	Q2	Search for non-existing ID 10	<code>node src/app.js</code>	Error (handled)	"No student found with this ID."
6	Q3	Execution order recorded	<code>node app.js</code>	Success	SYNC → MICROTASK → TIMER → FILE I/O → TIMER
7	Q4	Valid data	<code>node app.js valid</code>	Success	<code>SUCCESS: Processed 4 student records.</code>
8	Q4	Invalid marks	<code>node app.js invalid</code>	Error (handled)	Friendly validation message + cleanup
9	Q4	Missing file	<code>node app.js missing</code>	Error (handled)	"The data file was not found..." + cleanup
10	Q5	Add task	<code>node src/app.js add "Complete backend assignment"</code>	Success	Task added (ID: 1)
11	Q5	List tasks	<code>node src/app.js list</code>	Success	Table with summary
12	Q5	Complete task	<code>node src/app.js complete 1</code>	Success	Task marked as completed
13	Q5	Delete task	<code>node src/app.js delete 2</code>	Success	Task deleted

#	Question	Test	Command	Type	Result
14	Q5	Unknown task ID	<code>node src/app.js complete 99</code>	Error (handled)	"Task with ID 99 was not found."
15	Q5	Non-numeric ID	<code>node src/app.js complete abc</code>	Error (handled)	"Invalid task ID..."
16	Q5	Empty title	<code>node src/app.js add ""</code>	Error (handled)	"Task title cannot be empty..."
17	Q5	Duplicate title	<code>node src/app.js add "Complete backend assignment"</code>	Error (handled)	"A task with the title ... already exists."
18	Q5	Unknown command	<code>node src/app.js remove 1</code>	Error (handled)	"Unknown command" + help
19	Q5	Missing <code>.env</code>	<code>rename .env, run list</code>	Error (handled)	"CONFIG ERROR: Missing environment variable(s)..."

All the outputs shown in this report were produced by running the programs on Node.js v22. Values that depend on the computer — dates/times in Question 5, the number of progress dots and the milliseconds in Question 4 — will be slightly different on another system.

6.3 Screenshot Checklist

The following terminal screens should be captured and attached with this report:

#	Screenshot	Command(s)	Folder
1	Node.js version	<code>node -v</code>	any
2	Q1 legacy output	<code>node legacy.js</code>	question-1
3	Q1 modern output	<code>node app.js</code>	question-1
4	Q2 output	<code>node src/app.js</code>	question-2
5	Q2 folder structure (editor sidebar)	—	question-2
6	Q3 execution order	<code>node app.js</code>	question-3
7	Q4 success case	<code>node app.js valid</code>	question-4
8	Q4 failure – invalid data	<code>node app.js invalid</code>	question-4
9	Q4 failure – missing file	<code>node app.js missing</code>	question-4
10	Q5 dependency installation	<code>npm install</code>	question-5-task-manager
11	Q5 help	<code>node src/app.js help</code>	question-5-task-manager
12	Q5 add three tasks	three <code>add</code> commands	question-5-task-manager

#	Screenshot	Command(s)	Folder
13	Q5 list tasks	<code>node src/app.js list</code>	<code>question-5-task-manager</code>
14	Q5 complete, delete, list	<code>complete 1, delete 2, list</code>	<code>question-5-task-manager</code>
15	Q5 error - task not found	<code>complete 99, delete 99</code>	<code>question-5-task-manager</code>
16	Q5 error - invalid input	<code>complete abc, add ""</code>	<code>question-5-task-manager</code>
17	Q5 error - missing <code>.env</code> (optional)	<code>rename .env, list, rename back</code>	<code>question-5-task-manager</code>
18	Q5 folder structure + <code>.env.example</code>	—	<code>question-5-task-manager</code>

7. Conclusion

This assignment covered the core modern JavaScript concepts required for backend development with Node.js:

- **Question 1** showed that ES6+ features — `let / const`, arrow functions, template literals, destructuring, default values, optional chaining and nullish coalescing — make legacy code shorter, clearer and safer without changing its results.
- **Question 2** split a single-file student program into data, logic and main modules using ES Modules, which improves reusability, maintainability and teamwork.
- **Question 3** demonstrated how the Call Stack, Node.js APIs, the Microtask Queue, the Callback Queue and the Event Loop decide the actual execution order of synchronous and asynchronous code.
- **Question 4** built a reliable asynchronous processor using Promises, `async/await`, `try...catch...finally` and user-friendly error messages, which handled success and failure without crashing.
- **Question 5** combined all of these ideas in a working CLI Task Manager with ES Modules, asynchronous file storage, `.env` configuration, input validation and proper error handling.

Through this assignment I learned how to write backend JavaScript that is not only correct, but also well-organised, non-blocking and able to handle errors gracefully — skills that are essential for building real-world Node.js applications.

Submitted by: SAGAR KUMAR | **PRN:** SOE25BTAM29 | **Section:** A